ROB 498/599 · 3D Robot Perception · Fall 2026

L16 Latent dynamics

World models that predict what happens next under the robot's own actions, with the probability, recurrent networks, VAEs and reinforcement learning they rest on: planning with CEM, PlaNet, Dreamer and DreamerV3, and JEPA's prediction in representation space.

Mon Nov 2, 2026 • ANNOTATED build

Latent dynamics

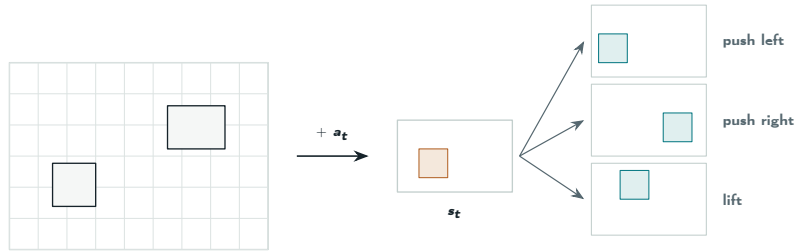
Every map so far has described the world as it is. A robot acts, so it also needs to know what the world will look like after it acts.

Unit 3, **world models and prediction**: from a static model of what is there to a predictive model of what happens next under the robot's own actions.

Today

1. **World models**, and the probability they are written in.
2. **Compounding error** and **planning** with a model: MPC, and CEM by hand.
3. **Learning a latent state:** likelihood, KL, autoencoders, the VAE.
4. **Recurrent networks**, then **PlaNet**'s recurrent state-space model.
5. **Reinforcement learning** from zero, then **Dreamer**.
6. **DreamerV3**'s fixes for one set of hyperparameters everywhere.
7. **Self-supervised learning**, then **I-JEPA**, and how later JEPAs extend it.
8. Two families compared, and where 3D fits.

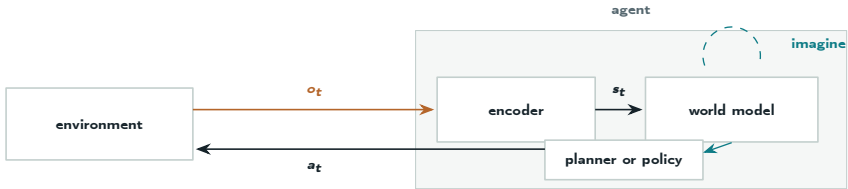
From what is there to what happens next



orange: the object as observed now; teal: where the model predicts it goes

- Units 1 and 2 built maps of the present: occupancy, TSDFs, Gaussians, scene graphs.
- A **world model** predicts the next state given the current one **and an action**.

Why a robot wants one

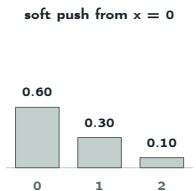


orange: the real observation; teal: the model's imagined futures

- **Plan:** try action sequences inside the model, run the best one.
- **Imagine:** train a policy on rollouts the model makes up, with no robot time.
- **Fewer real trials:** PlaNet matches a model-free agent's scores with 1,000 episodes instead of 100,000.
- **Safety:** a bad action tried in the model breaks nothing.

Probability

1 of 6: a random variable; illustrative numbers



Push a block on a table. It slides 0, 1 or 2 cm, never the same twice.

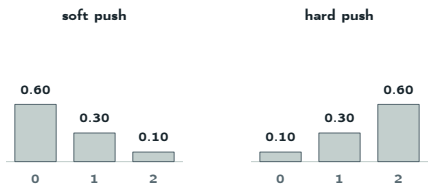
A **random variable** X is a quantity whose value is uncertain. Its **distribution** gives each value a probability:

$$p(X{=}0) = 0.6, \quad p(X{=}1) = 0.3, \quad p(X{=}2) = 0.1$$

Probabilities are ≥ 0 and sum to 1.

Probability

2 of 6: conditional distributions



$$p(x' \mid x, a): \quad p(x'{=}2 \mid x{=}0, \text{soft}) = 0.1, \qquad p(x'{=}2 \mid x{=}0, \text{hard}) = 0.6$$

A **conditional distribution** is one distribution per value of what we condition on. Each one sums to 1. A world model is exactly this: $p(s' \mid s, a)$.

Probability

3 of 6: expectation

$$\mathbb{E}[X] = \sum_x x\, p(x) \qquad \text{or, for continuous } x, \qquad \mathbb{E}[X] = \int x\, p(x)\, dx$$

Expected slide for the soft push, and for the hard push. Then average these 8 observed soft pushes: 0, 1, 0, 0, 2, 0, 1, 0.

Soft: $0 \cdot 0.6 + 1 \cdot 0.3 + 2 \cdot 0.1 = 0.5$ cm. Hard: $0 \cdot 0.1 + 1 \cdot 0.3 + 2 \cdot 0.6 = 1.5$ cm.

Sample average: $4/8 = 0.5$, the same as $\mathbb{E}[X]$ here.

The **expectation** is the probability-weighted average. With samples $x_1, \ldots, x_N$ from p, $\frac{1}{N} \sum_i x_i \approx \mathbb{E}[X]$.

Probability

4 of 6: variance and standard deviation

$$\mathrm{Var}[X] = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big], \qquad \sigma = \sqrt{\mathrm{Var}[X]}$$

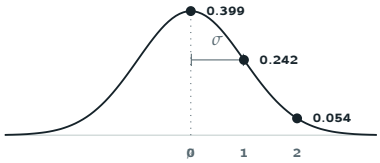
Variance and standard deviation of the soft push (mean 0.5).

$0.6(0-0.5)^2 + 0.3(1-0.5)^2 + 0.1(2-0.5)^2 = 0.45$, so $\sigma = 0.671$ cm.

The mean says where; the standard deviation says how spread out. A model that predicts both says how sure it is.

Probability

5 of 6: the Gaussian



$$\mathcal{N}(x;\ \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\Big(-\frac{(x-\mu)^2}{2\sigma^2} \Big)$$

- A distribution over a continuous x: a **density**, whose area is 1.
- Two numbers fix it: the mean μ and the standard deviation σ.

Evaluate $\mathcal{N}(x; 0, 1)$ at $x = 0,\ 1,\ 2$.

$\frac{1}{\sqrt{2\pi}} = 0.399;\ \ 0.399\, e^{-1/2} = 0.242;\ \ 0.399\, e^{-2} = 0.054.$

Probability

6 of 6: a Gaussian in many dimensions

With d independent dimensions (a **diagonal** Gaussian), the density is the product of d one-dimensional ones, so the log density is a sum:

$$\ln \mathcal{N}(\mathbf{z};\ \boldsymbol{\mu}, \operatorname{diag}(\boldsymbol{\sigma}^2)) = \sum_{i=1}^{d} \ln \mathcal{N}(z_i;\ \mu_i, \sigma_i^2)$$

$\ln \mathcal{N}(\mathbf{z}; 0, I)$ **for** $\mathbf{z} = (0.5, -1)$**.**

$-\frac{1}{2}(0.5^2 + -1^2) - 2 \cdot \frac{1}{2} \ln 2\pi = -0.625 - 1.838 = -2.463.$

PlaNet's stochastic state is a 30-dimensional diagonal Gaussian.

A world model, written down

If the state is known (joint angles, object poses), the model is a conditional distribution:

$$s_{t+1} \sim p(s_{t+1} \mid s_t,\ a_t)$$

From cameras the state is not observed, so predict the next frame from everything seen and done:

$$o_{t+1} \sim p(o_{t+1} \mid o_{1:t},\ a_{1:t})$$

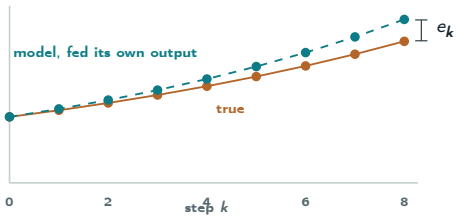
Latent models split this into an **encoder** $o_t \mapsto s_t$, a **transition** $p(s_{t+1} \mid s_t, a_t)$, and a **decoder** $s_t \mapsto o_t$.

What a world model is conditioned on

Conditioned on	What it can answer
past frames only	what usually happens next (video prediction). No use for choosing an action.
past frames **and actions**	what happens if *I* do this: planning, imagination training. Today's models.
a goal image or text	which actions reach the goal: the planner compares predictions with it.
a policy	what happens if I keep behaving like this: the value of a policy, without running it.

A one-step model, rolled out

illustrative numbers

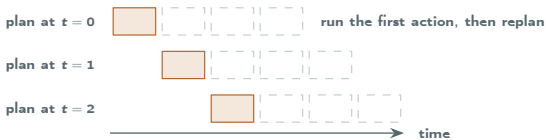


teal: produced by the model; orange: observed or true

- Trained on one step: given the true s_t, predict s_{t+1}.
- Used for k steps: the model's output becomes its next input.
- True system $x_{k+1} = 1.1\,x_k$, model 1.12. After 8 steps: off by 0.33.

Planning as search over action sequences

$$a^{\star}_{t:t+H-1} = \arg\min_{a_{t:t+H-1}} \sum_{k=0}^{H-1} c(\hat{s}_{t+k},\ a_{t+k}), \qquad \hat{s}_{t+k+1} = f(\hat{s}_{t+k}, a_{t+k}),\quad \hat{s}_t = s_t$$



dashed: planned in the model; orange: run on the robot

- **Model-predictive control (MPC):** solve for H actions, run only the first, observe, solve again.
- The cost c can be a predicted reward or a distance to a goal.
- The model need not be differentiable: the next methods only *evaluate* it.

Random shooting, then CEM

Random shooting: sample J sequences, score each in the model, run the first action of the best. Fine for small searches; PlaNet's cheetah has 6 action dimensions over 12 steps, a 72-dimensional search.

Cross-entropy method (CEM): sample near the best ones and repeat.

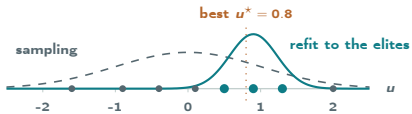
1. Gaussian over action sequences, $\mu = 0$, $\sigma = 1$.
2. Sample J sequences; score each in the model.
3. Keep the best K, the **elites**.
4. Refit μ, σ to the elites. Repeat I times.
5. Run the first action of μ; next step, start again from $\mathcal{N}(0, 1)$.

PlaNet: $H = 12$, $I = 10$, $J = 1{,}000$, $K = 100$: 120,000 predictions per action.

CEM by hand

1 of 2: one round, illustrative numbers

One action u, robot at $x = 0$, goal at 1, model $x' = x + u$. Cost $J(u) = (u-1)^2 + 0.25\,u^2$: reach the goal without pushing hard.



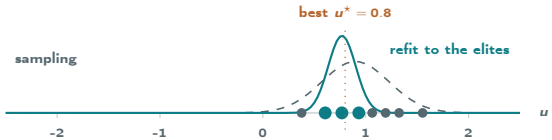
teal: the elites and the refitted Gaussian; orange: the true optimum

Eight draws from $\mathcal{N}(0,1)$: $-1.6,\ -0.9,\ -0.4,\ 0.1,\ 0.5,\ 0.9,\ 1.3,\ 2$. Score each, keep the best 3, refit μ and σ.

J: 7.400, 3.812, 2.000, 0.812, 0.312, 0.213, 0.513, 2.000. Elites $0.5, 0.9, 1.3$: $\mu = 0.90$, $\sigma = \sqrt{\frac{1}{3}(-0.4^2 + 0^2 + 0.4^2)} = 0.327$.

CEM by hand

2 of 2: the second round



teal: the elites and the refitted Gaussian; orange: the true optimum

Same eight z, now $u = 0.90 + 0.327\,z$. Which three win, and where does μ go?

u: 0.377, 0.606, 0.769, 0.933, 1.063, 1.194, 1.325, 1.553. Elites $0.606, 0.769, 0.933$, so $\mu = 0.769$, $\sigma = 0.133$: closer to $u^\star = 0.8$, and narrower.

Why plan in a latent space?

- PlaNet sees 64×64 RGB images: 12,288 numbers per frame. Its latent state is $200 + 30 = 230$ numbers, 53 times fewer.
- CEM needs 120,000 predictions per action: out of reach for images, affordable for small vectors.
- Pixels also carry what does not matter for the task: texture, lighting, the background. A latent can drop it.

The tool for learning a small state from images is the **variational autoencoder (VAE)**. It is trained by maximum likelihood with a KL penalty, so those come first.

Likelihood

illustrative numbers

Three measurements of the block's position: 0.8, 1.1, 1.4. Model: each is drawn from $\mathcal{N}(\mu, 1)$. Which μ explains them better?

$$L(\mu) = \prod_i \mathcal{N}(x_i;\ \mu, 1), \qquad \ln L(\mu) = -\tfrac{1}{2}\sum_i (x_i - \mu)^2 - \tfrac{3}{2}\ln 2\pi$$

$\ln L$ for $\mu = 1.1$ and $\mu = 0.5$.

$\mu = 1.1$: $\sum(x_i - \mu)^2 = 0.18$, $\ln L = 0.090 - 2.757 = -2.847$.

$\mu = 0.5$: $\sum(x_i - \mu)^2 = 1.26$, $\ln L = 0.630 - 2.757 = -3.387$. The first explains the data better.

The **likelihood** is the probability (density) of the observed data, read as a function of the model's parameters.

Maximum likelihood, and why MSE is one

Maximum likelihood: pick the parameters that make the data most likely. Here, set $\frac{d}{d\mu} \ln L = \sum_i (x_i - \mu) = 0$: $\mu =$ the mean $= 1.1$.

For one measurement x and a network's prediction $\hat{x}$, a unit-variance Gaussian gives

$$-\ln \mathcal{N}(x;\ \hat{x}, 1) = \tfrac{1}{2}(x - \hat{x})^2 + \tfrac{1}{2} \ln 2\pi$$

Evaluate it for $x = 3$, $\hat{x} = 2.5$.

$0.125 + 0.919 = 1.044$: squared error plus a constant.

So minimising mean squared error (L05) **is** maximum likelihood under a Gaussian. Cross-entropy (L05) is the same for a softmax.

Entropy

illustrative numbers

$$H(p) = -\sum_x p(x) \ln p(x)$$

Entropy of a fair coin $(0.5, 0.5)$**, and of a biased coin** $(0.9, 0.1)$**.**

Fair: $-2 \cdot 0.5 \ln 0.5 = \ln 2 = 0.693$ nats.

Biased: $-(0.9 \ln 0.9 + 0.1 \ln 0.1) = 0.325$ nats. Less uncertain, lower entropy.

Entropy measures how uncertain a distribution is: the average surprise $-\ln p(x)$ of its own samples. Zero when one outcome is certain.

From cross-entropy to KL divergence

illustrative numbers

$$H(p,q) = -\sum_x p(x) \ln q(x) \quad \text{(L05)}, \qquad \mathrm{KL}(p \,\|\, q) = H(p,q) - H(p) = \sum_x p(x) \ln \frac{p(x)}{q(x)}$$

p = fair coin, $q = (0.9, 0.1)$. Find $H(p,q)$, then $\mathrm{KL}(p \,\|\, q)$ and $\mathrm{KL}(q \,\|\, p)$.

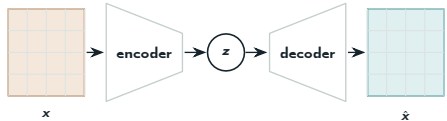
$H(p,q) = -0.5 \ln 0.9 - 0.5 \ln 0.1 = 1.204$; $\mathrm{KL}(p \,\|\, q) = 1.204 - 0.693 = 0.511$.

$\mathrm{KL}(q \,\|\, p) = 0.9 \ln \frac{0.9}{0.5} + 0.1 \ln \frac{0.1}{0.5} = 0.368$. Not the same: KL is not symmetric.

KL divergence: the extra surprise from using q when the data come from p. Always ≥ 0, and 0 only when $q = p$.

Autoencoders

1 of 2: the idea



orange: the input image; teal: the model's reconstruction

- An **encoder** maps an image x to a short code z; a **decoder** maps z back to $\hat{x}$.
- Train both on $\|x - \hat{x}\|^2$ by gradient descent (L05). The bottleneck forces z to keep what is needed to redraw x.
- **Self-supervised:** the target is the input itself, so no labels are needed.

Autoencoders

2 of 2: by hand, illustrative numbers

A linear autoencoder from 2 numbers to 1 and back. Encoder $z = 0.8\,x_1 + 0.6\,x_2$; decoder $\hat{x} = (0.8\,z,\ 0.6\,z)$.

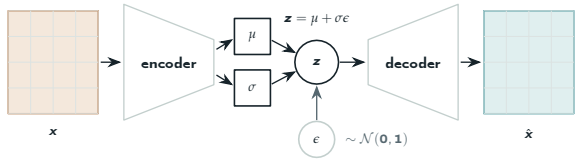
Encode $x = (2, 1)$, decode, and find the squared error.

$z = 0.8 \cdot 2 + 0.6 \cdot 1 = 2.2$. $\hat{x} = (1.76, 1.32)$. $\|x - \hat{x}\|^2 = 0.24^2 + -0.32^2 = 0.16$.

One number cannot hold two exactly; training moves the weights to lose as little as possible. Nothing says what a code *between* training codes decodes to.

Variational autoencoders

1 of 6: a distribution per input



orange: the input image; teal: the model's reconstruction

- The encoder outputs a mean $\mu(x)$ and a standard deviation $\sigma(x)$: a Gaussian $q(z \mid x)$ per input.
- Every code near μ must decode to something like x.
- **Analogy:** an autoencoder pins each image to one point on a map. A VAE gives each image a small district, so nearby points still decode to something.

Variational autoencoders

2 of 6: the reparameterisation, by hand

$$z = \mu + \sigma\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0,1)$$

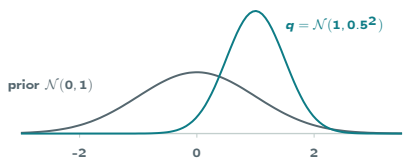
The encoder gives $\mu = 1$, $\sigma = 0.5$; the noise draw is $\epsilon = -0.4$. Find z, $\partial z/\partial\mu$ and $\partial z/\partial\sigma$.

$z = 1 + 0.5 \cdot (-0.4) = 0.8$. $\partial z/\partial\mu = 1$, $\partial z/\partial\sigma = \epsilon = -0.4$.

Sampling becomes a sum and a product, so backpropagation (L05) passes through it to the encoder.

Variational autoencoders

3 of 6: keep the clouds together



teal: the encoder's output for one image; grey: the shared prior

The encoder could cheat: shrink $\sigma \to 0$ and spread the means apart. So penalise the KL from each $q(z \mid x)$ to a shared **prior** $p(z) = \mathcal{N}(0, 1)$:

$$\mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0,1)\big) = \tfrac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \ln \sigma^2\right)$$

Variational autoencoders

4 of 6: the KL by hand

$$\mathrm{KL} = \tfrac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \ln \sigma^2\right)$$

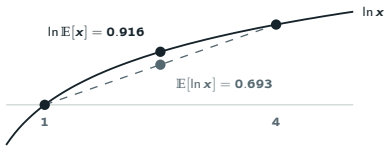
Evaluate it for $\mu = 1,\ \sigma = 0.5$; for $\mu = 0,\ \sigma = 1$; and for the cheat $\mu = 0,\ \sigma = 0.1$. What does each term punish?

$$\begin{aligned}
\mu = 1,\ \sigma = 0.5: &\quad \tfrac{1}{2}\,(1 + 0.25 - 1 - \ln 0.25) = \tfrac{1}{2}(0.25 + 1.386) = 0.818 \text{ nats} \\
\mu = 0,\ \sigma = 1: &\quad \tfrac{1}{2}(0 + 1 - 1 - 0) = 0 \\
\mu = 0,\ \sigma = 0.1: &\quad \tfrac{1}{2}\,(0 + 0.01 - 1 - \ln 0.01) = \tfrac{1}{2}(-0.99 + 4.605) = 1.808 \text{ nats}
\end{aligned}$$

μ^2 punishes a cloud far from 0; $\sigma^2 - 1 - \ln \sigma^2$ punishes one too narrow or too wide. The cheat costs the most.

Variational autoencoders

5 of 6: Jensen's inequality



$$\ln \mathbb{E}[x] \;\geq\; \mathbb{E}[\ln x]$$

$x = 1$ **or** 4**, equally likely.**

$\ln 2.5 = 0.916$; $\frac{1}{2}(\ln 1 + \ln 4) = 0.693$.

ln curves downward, so the log of an average is above the average of the logs.

Variational autoencoders

6 of 6: the ELBO

The model: draw a code $z \sim p(z)$, decode it with $p(x \mid z)$. Maximum likelihood wants $\ln p(x) = \ln \int p(x \mid z)\, p(z)\, dz$, but that integral is intractable.

Multiply and divide inside the integral by $q(z \mid x)$, then use Jensen's inequality.

$$\begin{aligned}\ln p(x) &= \ln \mathbb{E}_{q(z|x)}\left[\frac{p(x \mid z)\, p(z)}{q(z \mid x)}\right] \;\geq\; \mathbb{E}_q[\ln p(x \mid z)] - \mathbb{E}_q\left[\ln \frac{q(z \mid x)}{p(z)}\right] \\ &= \underbrace{\mathbb{E}_q\left[\ln p(x \mid z)\right]}_{\text{reconstruction}} - \underbrace{\mathrm{KL}\big(q(z \mid x) \,\|\, p(z)\big)}_{\text{keep the clouds together}}\end{aligned}$$

The **evidence lower bound (ELBO)**. Maximise it: decode well, and keep the codes near the prior.

The ELBO, by hand

one sample, illustrative numbers

One input $x = 1$. The encoder gives $q = \mathcal{N}(1, 0.5^2)$; the noise draw $\epsilon = -0.4$ gives $z = 0.8$ (two slides back). The decoder predicts $\hat{x} = z$, unit variance.

Estimate the ELBO from this one sample: reconstruction term, minus the KL.

Reconstruction: $\ln \mathcal{N}(1;\ 0.8, 1) = -\frac{1}{2}(0.2)^2 - 0.919 = -0.939$.

KL: 0.818 (the KL slide). ELBO $\approx -0.939 - 0.818 = -1.757$.

Training averages this over a batch and follows its gradient. The expectation is estimated with one sample.

Recurrent networks

1 of 2: a network with memory

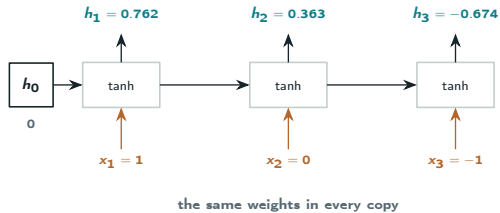
$$h_t = \tanh\left(Wh_{t-1} + Ux_t + b\right)$$

- An MLP (L05) sees one input. A **recurrent neural network (RNN)** reads a sequence one step at a time.
- The **hidden state** h_t is its memory: each step mixes the old memory with the new input.
- The same weights W, U, b are used at every step.

A world model needs this: the next frame depends on the whole history of frames and actions.

Recurrent networks

2 of 2: unrolled, by hand, illustrative numbers



orange: the input sequence; teal: the hidden states the network computes

Scalar RNN, $W = 0.5$**,** $U = 1$**,** $b = 0$**,** $h_0 = 0$**. Run it on** $x = 1, 0, -1$**.**

$h_1 = \tanh(1) = 0.762$; $h_2 = \tanh(0.5 \cdot 0.762 + 0) = 0.363$;
$h_3 = \tanh(0.5 \cdot 0.363 - 1) = -0.674$.

Gated memory

1 of 3: keep or overwrite, illustrative numbers

A plain RNN replaces its memory at every step. A **gated** network blends: the new memory is a point between the old memory and a **candidate**.

$$h_t = (1 - z)\, h_{t-1} + z\, \tilde{h}$$

z, the **update gate**, is between 0 and 1.



teal: the new memory at z = 0.2

Old memory $h_{t-1} = 0.8$. Candidate $\tilde{h} = \tanh(0.5 \cdot 0.8 - 1) = -0.537$, the plain RNN's answer. Blend with $z = 0,\ 0.2,\ 1$.

$z = 0$: 0.8 (keep). $z = 0.2$: $0.8 \cdot 0.8 + 0.2 \cdot (-0.537) = 0.533$. $z = 1$: -0.537 (overwrite).

Gated memory

2 of 3: the network sets the gate, illustrative numbers

$$z = \sigma(w_z\, h_{t-1} + u_z\, x_t)$$

Made-up weights $w_z = 1$, $u_z = 2$; $h_{t-1} = 0.8$. Find z and h_t for $x_t = -1$, then for $x_t = +1$.

$x_t = -1$: $1 \cdot 0.8 + 2 \cdot (-1) = -1.2$, $z = \sigma(-1.2) = 0.231$.
$h_t = 0.769 \cdot 0.8 + 0.231 \cdot (-0.537) = 0.491$: mostly kept.

$x_t = +1$: $1 \cdot 0.8 + 2 \cdot 1 = 2.8$, $z = \sigma(2.8) = 0.943$. Candidate $\tanh(0.5 \cdot 0.8 + 1) = 0.885$;
$h_t = 0.880$: mostly overwritten.

One small neuron, with a sigmoid (L05) so $0 \le z \le 1$. Training learns w_z, u_z: the network learns **when** to remember.

Gated memory

3 of 3: the reset gate, and the whole GRU

$$r = \sigma(w_r h_{t-1} + u_r x_t) \quad \text{reset gate}$$

$$z = \sigma(w_z h_{t-1} + u_z x_t) \quad \text{update gate}$$

$$\tilde{h} = \tanh\left(W\left(r\, h_{t-1}\right) + U x_t\right) \quad \text{candidate}$$

$$h_t = (1 - z)\, h_{t-1} + z\, \tilde{h} \quad \text{blend}$$

New piece: r sets how much of the old memory the candidate sees. r near 0 is a fresh start.

Same $h_{t-1} = 0.8$, $x_t = -1$, $z = 0.231$ from the last slide. Made-up $w_r = u_r = 1$.

$r = \sigma(1 \cdot 0.8 + 1 \cdot (-1)) = \sigma(-0.2) = 0.450$

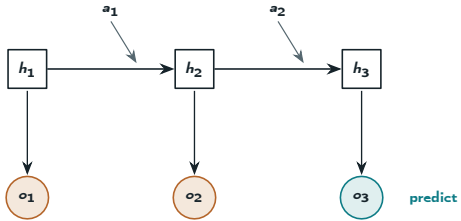
$\tilde{h} = \tanh(0.5 \cdot 0.450 \cdot 0.8 - 1) = \tanh(-0.82) = -0.675$

$h_t = 0.769 \cdot 0.8 + 0.231 \cdot (-0.675) = 0.459$

Plain RNN, same input: -0.537.

A latent dynamics model

1 of 4: deterministic only

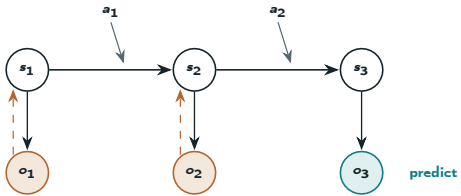


orange: observed frames; teal: the predicted frame

- A recurrent network: $h_t = f(h_{t-1}, a_{t-1})$, each h_t decoded to a frame.
- Remembers well, but predicts **one** future: it cannot say *the door might open or not*.

A latent dynamics model

2 of 4: stochastic only

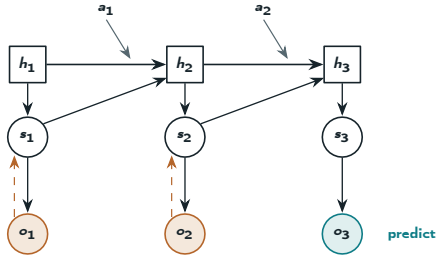


orange: observed frames and what is inferred from them; teal: predicted

- A state-space model: $s_t \sim p(s_t \mid s_{t-1}, a_{t-1})$, a Gaussian per step, so several futures can be represented.
- Dashed: an encoder infers s_t from the frame it saw, a VAE per step.
- All memory passes through one noisy sample per step, so long-range information is lost.

A latent dynamics model

3 of 4: both paths (RSSM)



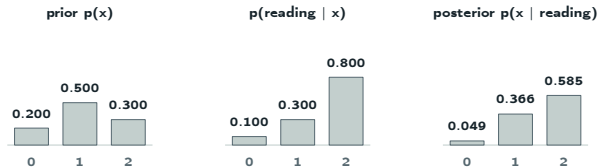
orange: observed frames and what is inferred from them; teal: predicted

$$h_t = f(h_{t-1},\ s_{t-1},\ a_{t-1}), \qquad s_t \sim p(s_t \mid h_t), \qquad o_t \sim p(o_t \mid h_t, s_t), \quad r_t \sim p(r_t \mid h_t, s_t)$$

The **recurrent state-space model**: h_t (a GRU, 200 units) carries memory; s_t (30-D diagonal Gaussian) carries uncertainty.

Bayes' rule: prior to posterior

illustrative numbers



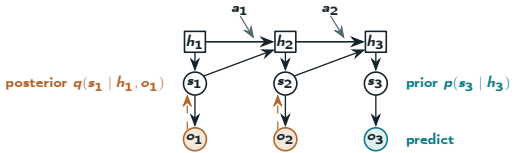
$$p(x \mid o) = \frac{p(o \mid x)\, p(x)}{p(o)}, \qquad p(o) = \sum_{x} p(o \mid x)\, p(x)$$

Before looking, the block is at $0, 1, 2$ with $0.2, 0.5, 0.3$. A sensor says "near 2", which it does with probability $0.1, 0.3, 0.8$ from each position. Where is the block now?

Products $0.02, 0.15, 0.24$, sum 0.41; divide: $0.049, 0.366, 0.585$.

A latent dynamics model

4 of 4: prior, posterior, loss



orange: observed frames and what is inferred from them; teal: predicted

- **Prior** $p(s_t \mid h_t)$: the prediction before seeing o_t. Used when imagining.
- **Posterior** $q(s_t \mid h_t, o_t)$: after seeing it. Used on real frames.
- **Analogy:** a navigator's dead reckoning (prior), corrected by a sighted landmark (posterior).

$$\mathcal{L} = \sum_t \Big(\underbrace{-\ln p(o_t \mid h_t, s_t) - \ln p(r_t \mid h_t, s_t)}_{\text{reconstruct}} + \underbrace{\mathrm{KL}\big(q(s_t \mid h_t, o_t) \,\|\, p(s_t \mid h_t)\big)}_{\text{predict what you will see}} \Big)$$

PlaNet: plan in the latent space

Hafner et al., ICML 2019

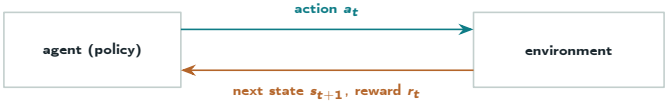
1. Filter the real frames into (h_t, s_t) with the posterior.
2. CEM over 12 actions, scored by summed predicted reward, rolled out with the **prior** only. Run the first action.
3. Every 100 updates, collect one more episode and add it to the data.



Mean return over six control tasks (Table 1), with a hundredth of the episodes. The paper also proposes **latent overshooting** (KL terms on multi-step predictions); its appendix says the final agent did not need it.

Reinforcement learning

1 of 9: learning from a score



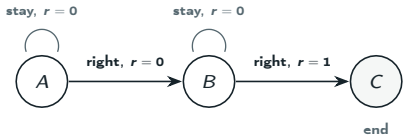
teal: what the agent chooses; orange: what the world returns

- In supervised learning (L05) every input comes with the right answer. Here the robot acts, and the world returns only a **reward**: a score, often long after the action that earned it.
- The robot has to work out for itself which of its actions led to the score. This is the **credit assignment** problem.

Analogy: learning a video game with no manual. You press buttons; the only feedback is the score.

Reinforcement learning

2 of 9: the pieces of the problem



- **State** s: what the robot knows now. Here A, B or C.
- **Action** a: what it can do. Here *right* or *stay*.
- **Transitions** $p(s' \mid s, a)$: how the world responds. This is exactly a **world model**.
- **Reward** $r(s, a)$: here 1 for reaching C, otherwise 0.

On a robot: the state is joint angles and an image, an action a motor command, the reward 1 when the block is in the bin.

Reinforcement learning

3 of 9: the return, and why discount

$$G_t = r_t + \gamma\, r_{t+1} + \gamma^2 r_{t+2} + \cdots = r_t + \gamma\, G_{t+1}$$

- The **return** adds up all the rewards still to come. The agent tries to make it large.
- The **discount** $\gamma < 1$ counts a reward k steps away as γ^k: sooner is worth more, and the sum stays finite.

weight γ^k	$k = 1$	$k = 10$	$k = 50$	$k = 100$
$\gamma = 0.9$	0.900	0.349	0.005	2.7×10^{-5}
$\gamma = 0.99$	0.990	0.904	0.605	0.366

On the chain, rewards $0, 0, 1$ give $G_0 = 0.9^2 = 0.81$. γ sets how far ahead the agent cares: about $1/(1-\gamma)$ steps.

Reinforcement learning

4 of 9: a policy, and why it should sometimes explore

$$\pi(a \mid s): \qquad \pi(\text{right} \mid A) = 0.8, \quad \pi(\text{stay} \mid A) = 0.2$$

- A **policy** says what to do in each state, as a probability for each action. In deep RL it is a network, the **actor**.
- **Exploration:** an agent that always takes its current best action never finds out whether another one is better.

Analogy: always ordering your favourite dish, or sometimes trying a new one. Trying costs a little tonight and can pay off on every later visit.

Reinforcement learning

5 of 9: value, the return you expect

$$v_\pi(s) = \mathbb{E}_\pi\big[\, G_t \mid s_t = s \,\big]$$

- The **value** of a state is the return to expect from there when following π, averaged over everything that could happen.
- It is a prediction about the future, made now. It lets the agent judge a state without playing it out.
- In deep RL it is a network, the **critic**.

Example: from s_0, two equally likely futures with returns 0.81 and 0, so $v(s_0) = 0.405$.

Analogy: a chess player who looks at a board and says *white is winning*, without playing the game to the end.

Reinforcement learning

6 of 9: the Bellman equation

$$v(s) = \sum_a \pi(a \mid s)\Big(r(s,a) + \gamma\, v(s')\Big) \qquad (s' \text{ the state } a \text{ leads to})$$

The value here is the reward for the next step plus the discounted value of where that step leads. Every state's value has to agree with its neighbours'.

On the chain, $v(C) = 0$ and the policy is *always right*. Find $v(B)$, then $v(A)$.

$v(B) = 1 + 0.9 \cdot 0 = 1$; $v(A) = 0 + 0.9 \cdot 1 = 0.9$. Solve from the end backwards.

A policy that stays put 20% of the time gets $v(B) = 0.976$ and $v(A) = 0.857$: hesitating costs value.

Reinforcement learning

7 of 9: learning values from experience (TD)

$$v(s) \leftarrow v(s) + \alpha \big(\underbrace{r + \gamma\, v(s')}_{\text{target}} - v(s) \big)$$

- Usually we cannot write the Bellman equation down: the transitions are unknown and the states too many.
- So after each real step, nudge $v(s)$ towards one reward plus the current guess for the next state: **temporal-difference (TD) learning**. A guess improved by another guess is called **bootstrapping**.

$\alpha = 0.5$	**start**	**after 1**	**after 2**	$\rightarrow$
$v(A)$	0	0	0.225	0.9
$v(B)$	0	0.5	0.75	1

Reinforcement learning

- **Critic** $v(s)$: learns values by TD.
- **Actor** $\pi(a \mid s)$: after each action, asks the critic whether it went better than expected.

$$\delta = r + \gamma\, v(s') - v(s) \qquad \delta > 0: \text{ make } a \text{ more likely}; \quad \delta < 0: \text{ less likely}$$

From B, with $v(B) = 0.75$: *right* gives $\delta = 1 - 0.75 = +0.25$, so raise it; *stay* gives $\delta = 0.9 \cdot 0.75 - 0.75 = -0.075$, so lower it.

Analogy: a coach who predicts how well each play will go, and a player who repeats the plays that beat the prediction.

Reinforcement learning

9 of 9: where the experience comes from

	Who produces the steps (s, a, r, s')
model-free RL	the real robot or simulator: millions of steps, slow, and wear on the hardware
model-based RL	a learned **world model**: cheap, thousands in parallel on a GPU, but carrying the model's errors

PlaNet used its model to **plan** at every step. Dreamer uses it to **practise**: an actor-critic trained on experience the world model imagines.

Dreamer: learn the behaviour in imagination

Hafner et al., ICLR 2020; figure from DreamerV3

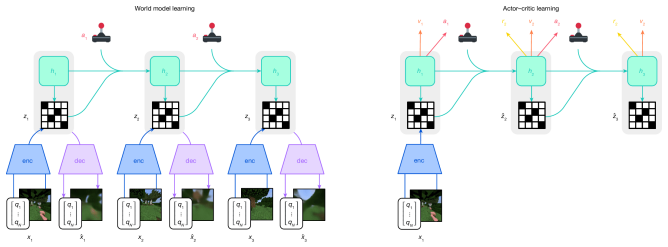


Figure: DreamerV3 (Hafner et al.), Nature 2025, Fig. 1, CC BY 4.0

- **Left, world model learning** on real frames x_t: encoder, recurrent state h_t, latent z_t, decoder. The RSSM of part 4.
- **Right, actor-critic learning** in imagination: from one real frame the model rolls forward to $\hat{z}_2, \hat{z}_3$ and predicts actions a, rewards r and values v. No image is decoded.

Dreamer's actor and critic

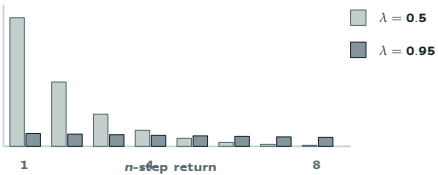
- **Critic** $v(s)$: regresses onto targets computed from imagined rewards and its own values.
- **Actor** $\pi(a \mid s)$: trained to make those targets large.
- The critic lets a 15-step imagined rollout count rewards long after it ends: stop, and add $\gamma^H v(s_{t+H})$.

Dreamer v1: 20 visual control tasks, average 823 after 5×10^6 steps, against PlaNet's 332 and model-free D4PG's 786 after 10^8.

λ-returns

1 of 2: what the critic is trained on

An n-step return uses n imagined rewards, then the critic: $R_t^{(n)} = r_t + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n v(s_{t+n})$. TD used $n = 1$.



- Small n: leans on the critic, which may be wrong (**bias**).
- Large n: leans on imagined rewards, which compound errors (**variance**).
- The λ-return weights $R^{(n)}$ by $(1-\lambda)\lambda^{n-1}$.

$$R_t^\lambda = r_t + \gamma\Big((1-\lambda)\, v(s_{t+1}) + \lambda\, R_{t+1}^\lambda\Big), \qquad R_H^\lambda = v(s_H)$$

Analogy: estimating a drive's arrival time. Ask the sat-nav now, or drive some legs and then ask it; the λ-return averages all of these.

λ-returns

2 of 2: by hand, illustrative numbers

A 3-step imagined rollout, $\gamma = 0.9$, $\lambda = 0.5$. Rewards $r_0, r_1, r_2 = 0, 0, 1$ (the return slide's); critic values $v(s_1), v(s_2), v(s_3) = 0.5, 0.8, 0.4$.

Work backwards from $R_3^\lambda = v(s_3)$ to R_0^λ. Compare with the one-step target and the full 3-step return.

$$
\begin{aligned}
R_3^\lambda &= 0.4 \\
R_2^\lambda &= 1 + 0.9\,(0.5 \cdot 0.4 + 0.5 \cdot 0.4) = 1.360 \\
R_1^\lambda &= 0 + 0.9\,(0.5 \cdot 0.8 + 0.5 \cdot 1.360) = 0.972 \\
R_0^\lambda &= 0 + 0.9\,(0.5 \cdot 0.5 + 0.5 \cdot 0.972) = 0.6624
\end{aligned}
$$

One-step: $0 + 0.9 \cdot 0.5 = 0.45$. Three-step: $0.9^2 \cdot 1 + 0.9^3 \cdot 0.4 = 1.1016$. The λ-return sits between.

What the world model imagines

DreamerV3, Fig. 3

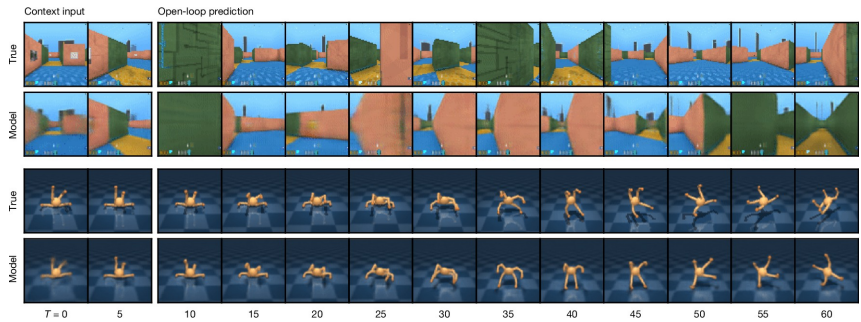


Figure: DreamerV3 (Hafner et al.), Nature 2025, Fig. 3, CC BY 4.0

Rows *True* and *Model*. The model sees frames at $T = 0$ and 5, then predicts open-loop to $T = 60$ without seeing another frame. These are decoded latents, so we can look at them.

DreamerV3: one configuration for every domain

Hafner et al., arXiv 2023; Nature 2025



Figure: DreamerV3 (Hafner et al.), Nature 2025, Fig. 2, CC BY 4.0

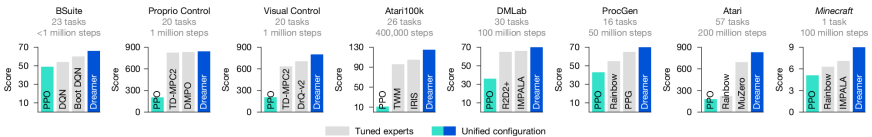


Figure: DreamerV3 (Hafner et al.), Nature 2025, Fig. 4, CC BY 4.0

The same hyperparameters on over 150 tasks in 8 domains. Bars: Dreamer (blue) against PPO and against the best expert method tuned for each benchmark (grey).

Why one configuration is hard

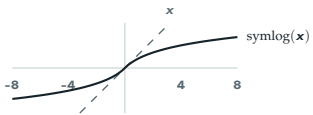
the four fixes

Rewards, returns and observations differ by orders of magnitude between domains, and so does every loss term. Earlier Dreamers retuned for each domain. DreamerV3 adds four fixes:

Fix	What it handles
symlog, two-hot	targets of unknown size: a reward of 0.01 and one of 10,000
free bits, KL balancing	the world model's KL swamping, or vanishing against, reconstruction
return normalisation	the actor's entropy bonus mattering too much or too little
1% unimix	categorical distributions going deterministic

Symlog, by hand

$$\operatorname{symlog}(x) = \operatorname{sign}(x)\ln(|x|+1), \qquad \operatorname{symexp}(x) = \operatorname{sign}(x)\left(e^{|x|}-1\right)$$

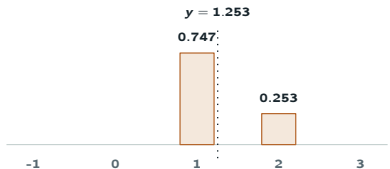


- Predict $\operatorname{symlog}(y)$ with a squared error; read out symexp of the output.
- **Analogy:** decibels, which put a whisper and a jet engine on one scale.

symlog of 1000, −5, 0.1? Predicting 0 for 1000: squared error with and without symlog?

$\ln 1001 = 6.909$, $-\ln 6 = -1.792$, $\ln 1.1 = 0.0953$. Squared error 1,000,000 raw, 47.7 after symlog.

Two-hot, by hand



orange: the training target the network's softmax is fitted to

- Rewards and values are predicted as a **classification** over bins in symlog space: a softmax, trained by cross-entropy (L05).
- The target puts weight on the two bins either side of $\mathrm{symlog}(y)$, so that its weighted mean is exactly the target.

Encode a return of 2.5 over unit-spaced bins (illustrative). Check the weighted mean.

$y = \ln 3.5 = 1.2528$, between bins 1 and 2: weights $2 - 1.2528 = 0.7472$ and 0.2528. Mean $0.7472 \cdot 1 + 0.2528 \cdot 2 = 1.2528$. Prediction: symexp of the predicted mean.

Free bits and KL balancing

The RSSM's KL, split into two terms that train different networks (sg is a stop-gradient):

$$\mathcal{L}_{\text{dyn}} = \max\Big(1,\ \mathrm{KL}\big(\mathrm{sg}(q(s_t \mid h_t, o_t)) \,\|\, p(s_t \mid h_t)\big)\Big) \quad \text{trains the prior}$$

$$\mathcal{L}_{\text{rep}} = \max\Big(1,\ \mathrm{KL}\big(q(s_t \mid h_t, o_t) \,\|\, \mathrm{sg}(p(s_t \mid h_t))\big)\Big) \quad \text{trains the encoder}$$

- **Balancing:** $\beta_{\text{pred}} = 1$, $\beta_{\text{dyn}} = 1$, $\beta_{\text{rep}} = 0.1$. The prior moves towards the encoder ten times harder than the reverse.
- **Free bits:** below 1 nat, no gradient. The KL slide's 0.818 nats would be left alone.

Return normalisation, by hand

illustrative returns

The actor maximises returns plus an entropy bonus. For one bonus weight to suit every task, returns are divided by their spread, but only when the spread is large:

$$\frac{R_t^\lambda - v(s_t)}{\max(1,\ S)}$$

$$S = \mathrm{Per}(R^\lambda, 95) - \mathrm{Per}(R^\lambda, 5)$$



Two batches of 20 returns: fifteen 0s and $0.1, 0.2, 0.3, 0.5, 0.8$; and $100, 120, \ldots, 480$. Find S and the divisor for each.

Sparse: $S = 0.515 - 0 = 0.515$, divisor $\max(1, S) = 1$: left alone. Dense: $S = 461 - 119 = 342$, divisor 342.

Which fixes matter

DreamerV3, Fig. 6

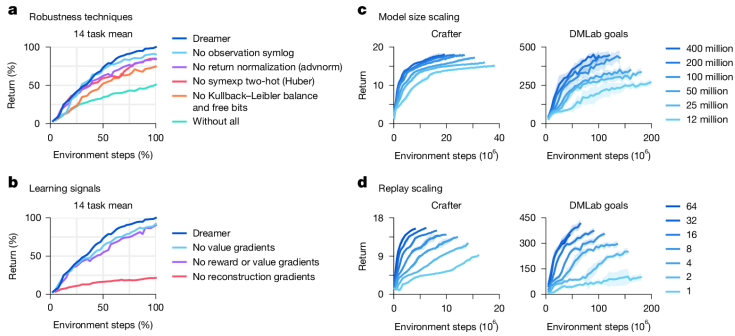


Figure: DreamerV3 (Hafner et al.), Nature 2025, Fig. 6, CC BY 4.0

a: remove one fix at a time. **b:** remove a learning signal. **c, d:** larger models and more replay learn faster.

Minecraft diamonds

DreamerV3, Fig. 5

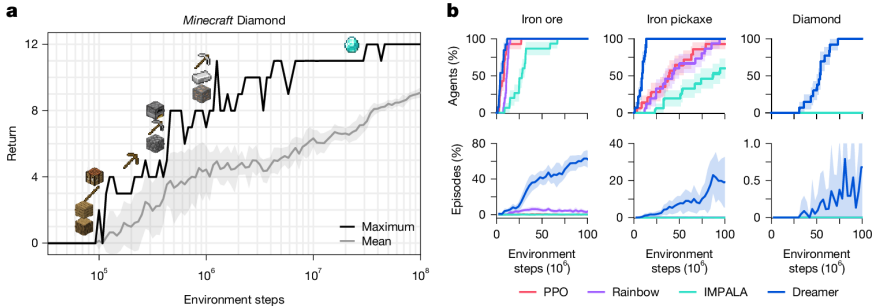


Figure: DreamerV3 (Hafner et al.), Nature 2025, Fig. 5, CC BY 4.0

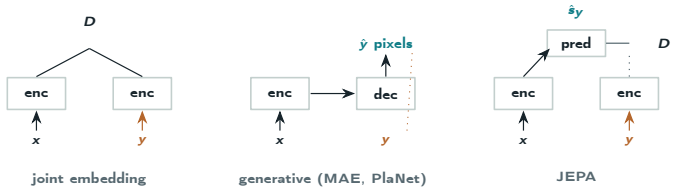
A diamond needs 12 items in order, from sparse rewards, in a new random world each episode. Dreamer is the only method in **b** to reach it, without human data, on 1 GPU for 9 days.

Self-supervised learning

- **Supervised** (L05): every input comes with a human label.
- **Self-supervised:** hide part of the data and predict it from the rest. The labels come from the data itself.
- Examples so far: the autoencoder predicts its own input; the world model predicts the next frame.

The goal is usually the **representation**: an encoder whose features are useful for other tasks, learned from unlabelled images and video.

Why predict pixels at all?

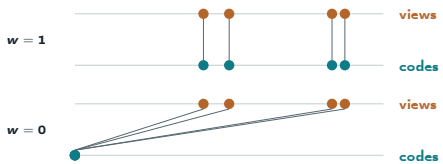


orange: the target y (real data); teal: the model's prediction; D: the distance compared

- PlaNet and Dreamer learn their state by **reconstructing frames**. Most of a frame is irrelevant to control: leaves, carpet texture, glare.
- A pixel loss spends capacity on all of it, and punishes a sharp but slightly wrong future more than a blurry average.
- **Joint-embedding predictive architecture (JEPA):** encode both, and predict the target's *code* from the context's.

Representation collapse

a two-point example, illustrative numbers



orange: the input views; teal: the codes $f(x) = w\,x$

Two images, two views each: $(1, 1.2)$ and $(2, 2.1)$. Encoder $f(x) = w\,x$, both sides. Loss: make the two views' codes agree.

Loss $\sum(w\,x - w\,y)^2$ for $w = 1$ and $w = 0$.

$w = 1$: $0.04 + 0.01 = 0.05$. $w = 0$: 0. In general $0.05\,w^2$: gradient descent drives $w \to 0$.

At $w = 0$ every image gets the same code: zero loss, and a representation that says nothing. Pixel targets cannot collapse like this, because the data are fixed.

What a linear probe measures

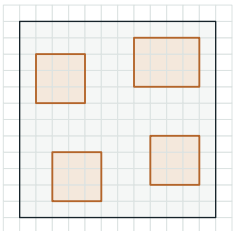
illustrative numbers



- Freeze the pretrained encoder. Train only a **linear** classifier, softmax regression from L05, on its features with labels.
- Left: the line $y = x$ separates the classes, 4/4 correct. Right: collapsed features, all at one point; the best is 2/4, chance.
- Probe accuracy measures how much class information the features hold in an easily readable form.

I-JEPA

1 of 3: what to hide (Assran et al., CVPR 2023)

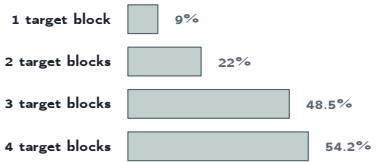


orange: target blocks, the real patches to be predicted; grey: context

- Cut the image into patches, as a ViT does (L10).
- **Targets:** 4 blocks, each covering 15 to 20% of the image.
- **Context:** one large block, 85 to 100%, with the targets cut out.
- Targets large enough to need semantics (a dog's head), context big enough to infer them from.

I-JEPA

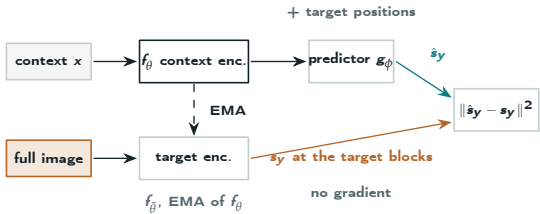
2 of 3: how many targets



- I-JEPA, Table 10: ViT-B/16, 300 epochs, linear probe with 1% of ImageNet labels.
- Only the number of target blocks changes.
- One block: 9.0%. Four: 54.2%.

I-JEPA

3 of 3: two encoders and a predictor (redrawn from Fig. 3)



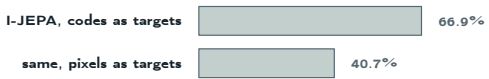
teal: the predicted codes; orange: target codes computed from the real image

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^{M} \sum_{j \in B_i} \left\| \hat{s}_{y_j} - s_{y_j} \right\|_2^2, \qquad \bar{\theta} \leftarrow \tau\, \bar{\theta} + (1 - \tau)\, \theta$$

The **target encoder** $\bar{\theta}$ is a slow copy of the context encoder θ and gets no gradient, so training cannot drag it to a constant. τ rises from 0.996 to 1: with θ still, half the gap closes in 173 steps.

What predicting codes buys

I-JEPA, Tables 1 and 7



- Same ViT-L/16, linear probe with 1% of ImageNet labels: the target is the only change.
- Full ImageNet, linear probe: ViT-H/14 79.3% after 300 epochs, against MAE's 77.2% after 1600.
- Under 1,200 GPU hours to pretrain the ViT-H/14.

How JEPA is being extended

follow-up papers

Paper	Data	What is hidden and predicted	What changes
I-JEPA (2023)	images	codes of large image blocks	the original: codes, not pixels
MC-JEPA (2023)	video frames	codes, plus optical flow between frames	learns motion and content in one encoder
A-JEPA (2023)	audio	codes of masked spectrogram patches	the same recipe on sound
V-JEPA (2024)	video	codes of space-time tubes	time; features beat pixels on video too
Point-JEPA (2024)	point clouds	codes of hidden groups of points	3D input
V-JEPA 2 (2025)	video, then robot video	next-frame codes given the arm's action	actions: a world model; plans with CEM

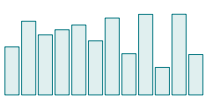
The recipe stays fixed: mask, predict codes, slow target encoder. What changes is the data, and in V-JEPA 2, the actions.

Two families of latent world model

	Reconstruction (PlaNet, Dreamer)	**Latent prediction (JEPA)**
State learned by	decoding frames (ELBO)	predicting the codes of hidden parts
Collapse prevented by	pixel targets are fixed	EMA target, stop-gradient
Actions	in the model from the start	none in I-JEPA; added in V-JEPA 2
Decides by	CEM (PlaNet) or a trained actor (Dreamer)	I-JEPA does not act; V-JEPA 2 plans with CEM
Data	one task's own experience, with reward	unlabelled web images or video
Weak point	capacity spent on irrelevant pixels	no reward; actions only added later

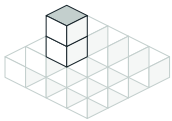
Neither family's state has units or geometry.

Where 3D fits



latent s_t: a vector

no axes, no units



3D state: occupancy, poses

metres, free space, contact

teal: a learned latent state

- None of these models has an explicit 3D state: depth, occlusion and contact are inferred from 2D frames.
- A 3D state gives **units** (metres), **checks** (free space, collision, L07) and a **frame** shared with the actions.
- The cost: the map must be built, and predicted forward too.

Wednesday, L17

Occupancy forecasting and 4D scene models: predicting the future of a 3D state, a grid in metres.

Learn it again, differently

other explanations

- Lilian Weng, From Autoencoder to Beta-VAE: the VAE and the ELBO step by step.
- Sutton and Barto, Reinforcement Learning: An Introduction, free online: Chapters 3 (MDPs, Bellman) and 6 (TD).
- Google Research posts on PlaNet and Dreamer; the DreamerV3 page has videos of every domain.
- Ha and Schmidhuber, World Models (2018): a car racer trained inside its own dream.
- Meta's posts on I-JEPA and V-JEPA 2, and the V-JEPA 2 code.
- Hou et al., World Model for Robot Learning: A Comprehensive Survey (2026), with a project page.

What you should be able to do

- Compute an expectation, a Gaussian density, a log-likelihood, an entropy and a KL; say why MSE is a Gaussian likelihood.
- Explain why a one-step model drifts when rolled out; run two rounds of CEM by hand.
- Run a VAE forward by hand: encode, reparameterise, decode, ELBO. Derive the ELBO.
- Step an RNN and a GRU by hand; draw the RSSM and say what its prior and posterior are.
- Explain return, value and the Bellman equation; solve it on a small chain; say what TD learning and actor-critic do; compute a λ-return.
- Compute symlog, a two-hot target and DreamerV3's return scale, and say which problem each fixes.
- Show collapse on a toy example; compute an EMA update; explain what a linear probe measures; say how I-JEPA chooses its masks and why it predicts codes; name what later JEPAs change.

Next

L17 Wed Nov 4

- **L17, Wed Nov 4: 4D and occupancy forecasting.** Response paper: Occupancy Networks.
- **Responses due Sun Nov 8:** DreamerV3 and Occupancy Networks.
- Also assigned for Unit 3: PlaNet, I-JEPA, V-JEPA, V-JEPA 2, and the survey.
- **Seminars, student-led:** Mon Nov 9 (Unit 2) and Wed Nov 11 (Unit 3). I am at CoRL that week.